

Note: the content here was not finished. I do not remember how much of this is fully up to date (some of it is not.) If I come back to this project this could get finished maybe

Unnamed: the inevitable sequel to LIN(E/u)mber

Based on the same idea of the program itself being used as its own memory, I have created a new and (hopefully) improved version of this very stupid language.

Notable Changes

- Standardization of inputs, addition of pointers and literals
 - In order to make instructions consistent, inputs have been split into multiple different formats, making the language more versatile and easier to understand. Actually, some of the new syntax might make this harder to understand.
- Syntax rework
 - All inputs are now placed to the right of the instruction, making 0-input instructions simpler to write and allowing for some instructions to receive more than 2 inputs.
- Instruction input changes
 - Some instructions have had inputs added or changed to allow for more versatile use cases.
- Instruction name changes
 - Several instructions have had their names changed in order to make them easier to remember and differentiate from each other.
 - Also for some reason I changed it so that now the built in methods are called instructions not really sure why I did this but I did.
- New instructions
 - Multiple new instructions have been added, making certain tasks easier to accomplish.
 - `match` has been introduced to make comparing values for equivalency easier.
 - `modify` has been introduced, now allowing the instructions themselves to be modified.
 - `shift` has been introduced, but I honestly forgot why.
 - `output` has been introduced, allowing for methods to output more than one value or have additional operations between producing an output and exiting the method.
 - `string` has been introduced, allowing for large sets of text to be printed to the console while using only one line.
- Removed instructions

- Some of the old instructions have been removed, but no functionality has been lost.
- `typeMethod` has been removed in favor of the new `string` instruction, which when combined with the new syntax makes printing text reasonably okay.
- `exit` has been removed. In its place, it is now possible to use the `return` instruction with no inputs to accomplish the same task.
- Instruction capitalization
 - Instructions no longer begin with capital letters. Other keywords, such as ‘Method’ have not been changed.

General Overview

This section is intended to act as an introduction to the language and cover everything one would need to know to create a program in it! (assuming anybody would want to to begin with)

[put background information type thing here)

The basic idea behind this language is that data is not stored using a conventional method such as variables, arrays, or stacks. Instead, the program itself serves as a container for any data that it needs to store.

Consider the following code segment (note: `doSomething` and `doNothing` are not valid instructions):

```
doSomething 1
doNothing 10
```

You would most likely expect that each instruction would use its corresponding input (the value written to the right of the instruction) as its input— if `doSomething` was meant to print its input to the console, you would expect it to print ‘1’.

This is not how this language works.

Instead, the ‘1’ input into `doSomething` means “the first value on *line 1*.” The first line with an instruction in this language is considered *line 0*, so in this situation, `doSomething 1` would take the value on line 1, which is 10, then use *that* (10) as its instruction.

Here’s a slightly longer example:

(Line numbers are shown in this example for clarity, but are not shown in future examples. For following examples, it can be assumed that all programs begin on *line 0*)

```
0 print 1
1 print 2
2 print 3
```

```
3 print 4
4 print 0
```

Firstly, the `print` instruction will display a single value onto the console as a base-10 integer and will not place a new line, space, or any other characters onto the console with it. Here is how this program would be executed:

Line	Process	Line Output	Full Output
0	Prints the first value shown on line 1	2	2
1	Prints the first value shown on line 2	3	23
2	Prints the first value shown on line 3	4	234
3	Prints the first value shown on line 4	0	2340
4	Prints the first value shown on line 0	1	23401

Note that each value is used both as a reference to another value and as the value being referenced. Any values typed can be used this way.

```
print 0
```

In this example there is only a single line (line 0.) Here is how it would be executed:

Line	Process	Line Output	Full Output
0	Prints the first value shown on line 0	0	0

This example shows how the same value can reference itself. In practical purposes it is very rare that this situation would occur, but it is useful to note its existence.

Up to this point, values that have been shown will always be able to act similarly to a reference to a variable. But what if you just want a constant value? Well that's where the newly introduced `#` syntax comes in handy! Here is an example:

```
print #300
```

Obviously this program does not contain 301 lines, but that's okay! `300` does not mean the same thing as `#300`. By adding the `#`, the program will now interpret this directly as *the value 300*. Here is how it would be executed:

Line	Process	Line Output	Full Output
0	Prints the value 300	300	300

This makes it easy to manage values that will only be used in specific places, as they can be stored in the same location in the program as they are used.

Here is the longer example used before with `#` placed before each value:

```
print #1
print #2
print #3
print #4
print #0
```

Line	Process	Line Output	Full Output
0	Prints the value 1	1	1
1	Prints the value 2	2	12
2	Prints the value 3	3	123
3	Prints the value 4	4	1234
4	Prints the value 0	0	12340

The `type` method acts very similarly to the `print` method, but will print the corresponding unicode character for the value input (in base 10). For example, this is how you could print a single capital letter A:

```
type #65
```

Line	Process	Line Output	Full Output
0	Print the unicode character with decimal value 65	A	A

Originally, this was how strings of text would be printed, requiring each character to be written as a unicode value and requiring an entire line of code per character. However, while designing this language, I decided that I wanted the language to be unusual, but not needlessly difficult, so I decided to implement some new additions to make printing string significantly easier.

First of all, there is now `"` syntax, which will turn a single character to its immediate right into a unicode value. This is done before the program runs, meaning that during execution it will be treated just the same as any written value, while making programming slightly easier. Note that the `"` still requires a `#` or else it will take a value from another line, which is something that you would virtually never want to do with `"` syntax.

This example shows how values can be output using all of the methods shown so far:

```

print #”A
print #65
print 0
type #”A
print #65
print 4

```

Line	Process	Line Output	Full Output
0	Prints the value 65 (A)	65	65
1	Prints the value 65	65	6565
2	Prints the value shown on line 0	65	656565
3	Print the unicode character with decimal value 65 (A)	A	656565A
4	Print the unicode character with decimal value 65	A	656565AA
5	Prints the unicode character shown on line 4	A	656565AAA

Syntax

Instructions:

```
[instruction] [value 1] [value 2] ...
```

[instruction] is required and determines what method this line refers to.

[value 1] and [value 2] may or may not be required depending on the instruction used. These values must be in the form of integers. Instructions may require 0 pointers or more with no limit. Excess values will be ignored.

The following are all legal examples of different amounts of inputs:

```

return
print #10
string *#” H e l l o

```

Instructions are also able to receive more inputs than they will use. In these situations, the addition inputs will be ignored by the instruction but still accessible in the memory. The following are examples of this:

null 0 1 2 3 4 5 (null instructions will always do nothing)

print 3 2 1 (the first value will be used, the rest ignored)

add 8 9 7 3 4 5 2 0 1 6 (the first three values will be used, the rest ignored)

Comments:

Beginning a line with / will mark the line as a comment. Comment lines will be ignored and will not count towards line counts

Value Syntax:

Values act as integers in method inputs. Values can be represented by pointers, advanced pointers, or literals.

[line]

Pointers are written as a single integer and represent the first value on a given line number. Pointers are used to reference a value in another line, which can be both read and written over. When referenced by another pointer, a pointer will act as a value.

5 ⇒ the first value on line 5

[\$line]:[index]

Advanced pointers are indicated with a \$. An advanced pointer represents the [index]th value on a given line number, with the first value considered the 0th value. The [index] portion of the advanced pointer is not part of a value and cannot be written over. *This is subject to change.*

\$10:0 ⇒ the first value on line 10

\$8:3 ⇒ the third value on line 8

#[value]

Literals are indicated with a #. Literals represent a value directly. When a literal is used as an input that would be modified, it will point to itself. When referenced by a pointer, will act as a value. When a literal is rewritten to another value, it will remain a literal, not become a pointer. (acts as a value with a pointer to itself)

#65 ⇒ represents the value 65. References its own line when used as a pointer specifically.

~[value]

Tildes will add the current line's value onto any value to the right of them before the program runs. These are useful for long programs where it is hard to keep track of what line everything is on. Note that the tilde is applied before running and will not adjust if a value stemming from a tilde moves to a new line.

(on line 30) ~10 ⇒ 30 + 10 = 40 ⇒ 40

(on line 23) $\# \sim 2 \Rightarrow 23 - 2 = 21 \Rightarrow \#21$

`"[c]`

Characters are converted into their corresponding number in unicode before the program runs and can be combined with other syntax above. Characters can be useful to simplify outputting text. Non-integers not marked as a character will result in an error

`"A` $\Rightarrow$ represents a pointer marking line 65

`$10."A` $\Rightarrow$ represents a pointer marking the 66th (index 65) input on line 10

`#"A` $\Rightarrow$ represents the value 65

`*[syntax]`

Universal Modifiers may be placed between the instruction and the first input or between inputs. Before the program runs, all syntax connected to the right of the asterisk will be applied to all to the right of the area on the same line.

<code>*# 5 10 6 8</code>	<code>#5 #10 #6 #8</code>
<code>*" A ! 0</code>	<code>"A "!" "0</code>
<code>1 2 *# 3 4</code>	<code>1 2 #3 #4</code>

`@[value1]:[value2]`

Swap syntax may be placed between the instructions and the first input. Swap syntax should be placed before universal syntax. Swap syntax will replace all instances of the first value on a given line with the second value. This applies to values before the program runs, but after other operations such as `~` and `"`.

<code>@10:0 0 5 10</code>	<code>0 5 0</code>
<code>@10:0 #0 #5 #10</code>	<code>#0 #5 #10</code>
<code>@10:0 \$10:5 \$5:10</code>	<code>\$0:5 \$5:0</code>

This is useful for adding characters that are difficult or impossible to otherwise represent (such as line feeds, spaces, and tabs) to `string` instructions or similar methods.

<code>string *#" H e l l o , _ W o r l d !</code>	In this string format spaces cannot be types because they are acting as whitespace. Similarly, trying to place a line feed would split the line into two different lines.
---	---

string @"_:32 *#" H e l l o , _ W o r l d !	In this string, the @"_:32 will replace all '_' characters with 32, the value that corresponds with space.
---	--

All above syntax must go in the following order (going in left to right order:

Applier		Data Type	Modifier	
@	*	#, \$	~	"
Swap	Universal	Literal, Advanced	Relative	Character

Method Initialization:

Method [method name]

Method is required to mark that a new method is being declared.

[method name] is required. To reference this method in the future, write it the same and the instruction syntax (as seen above.)

The first line after a method declaration is considered line 0 rather than line 1, with each line after being the next value (ie. 1,2,3,4.)

A method will only end when the next method is declared.

Special Methods:

Method main declares the main method, which will indicate where the program will start from. The main method can be placed anywhere in the program and is required for the program to run.

Method exit declared the exit method. Unlike other methods, this method cannot be executed or have instructions written in it. Instead, it marks where the end of your program is.

This method is required to run the program and anything written below this method will be ignored.

File formats

Programs can be written with any of the following formats:

Extension	Expanded	Use
.ln	Line Number	Interpreted as line number program
.lnd	Line Number Debug	Interpreted as line number program with

		extra debug information shown in console
(any other)		Interpreted as line number program because I didn't want to prevent the program from reading other files

Instructions

null (was :)

`null [p]...`

Null instructions do nothing on their own but can be used to hold values. A null instruction may have multiple input values.

print

`print [p]`

Outputs the value referenced by the input to the console as an integer

```
Method main
null 16
print 0
print #1
Method exit
```

Output: 161

Output '16' because the pointer on line 1 references line 0 (value of 16)

Output '1' because the literal on line 2 references itself (value of 1)

printMethod

`printMethod`

TBD if this instruction will be implemented

type

`type [p]...`

Outputs the value referenced by the input(s) to the console as unicode characters. Any amount of inputs may be provided to print single characters or strings of varying length.

```
Method main
null 65
type 0
type #33
Method exit
```

Output: A!

Output 'A' because the pointer on line 1 references line 0 (value of 65 = unicode A)

Output '!' because the literal on line 2 references itself (value of 33 = unicode !)

All of the following will output "Hello". Note that spaces are still required between characters

Tedious	Slightly better	Easiest solution
<pre>type #"H type #"e type #"l type #"l type #"o</pre>	<pre>type #"H #"e #"l #"l #"o</pre>	<pre>type *#" H e l l o</pre>

typeMethod

typeMethod

TBD if this instruction will be implemented

compare

compare [p] [q] [dest]

If the first value referenced is more than the second value referenced, jump to the line of code that the destination points to and continue from there. Because literals point to themselves, placing a literal as the destination will return to the same line, forcing the program into an infinite loop. Do not put a literal here.

```
Method main
null 10 8
compare $0:0 $0:1 3
type #"A
type #"B
Method exit
```

Output: B

The first input points to the first value in line 0 and the second input points to the second value in line 0.

In this situation the first value is greater ($10 > 8$) so the program moves directly to line 3, ignoring line 2 in the process.

Match (new)

Match [p] [q] [dest]

If both values referenced are equal, jump to the line of code that the destination points to and continue from there. As with the `compare` method, do not put a literal as the third input.

```
Method main
null 6 8 6
match $0:0 $0:1 3
match $0:0 $0:2 4
print #10
print #20
Method exit
```

Output: 20

The first match statement compares 6 and 8, which are not equal, so nothing happens. The second match statement compares 6 and 6, which are equal, so the program jumps to line 4, printing only 20.

goto *(was travel)*

`goto [dest]`

Moves the program directly to the line referenced by a pointer input.

```
Method main
goto 2
type #65
goto $4:0
type #66
type #67
Method exit
```

Output: C

The first 2 print statements are each skipped by a goto statement. The goto statement on line uses an advanced pointer to reference a specific index on the line, which is ignored by the goto statement.

transport

`transport [source] [dest]`

Moves any value stored in the source location to the destination location. This will preserve the value type (pointer, advanced pointer, literal) in the location but modify the value that it stores.

```
Method main
null 0 65
transport $0:1 2
type #0
Method exit
```

Output: A

This program moves the 65 stored as the second index (index 1) of line 1 to line 3. It then prints 'A' because the value stays as a literal.

Modify *(new)*

`Modify [instr] [dest]`

Replaces the instruction at the destination line with the instruction with the id corresponding to the instr value. Replaces the instr with a `null` instruction (ID=0) if instr is out of range

shift *(new)*

`shift [dest]`

Shifts all values on a line right by 1. The rightmost value then moves to the left

int *(was snag)*

`int [dest]`

char *(was ascii)*

`char [dest]`

Using a literal will result in the line storing the input in itself!

add

```
add [p] [q] [dest]
```

subtract

```
subtract [p] [q] [dest]
```

multiply

```
multiply [p] [q] [dest]
```

divide

```
divide [p] [q] [dest]
```

mod

```
mod [p] [q] [dest]
```

return *(was exit, was: return)*

```
return [p] [lines=1]
```

output *(new)*

```
output [p] [lines=1]
```

Same as return, does not exit

skip

```
skip [lines]
```

end

```
end
```

string *(new)*

```
string [c1]...
```

prints a series of characters to the console. A string acts similarly to a series of type instructions. A string can be written more easily by making use of both characters and universal syntax.

All of the following will output "Hello". Note that spaces are still required between characters

Tedious	Slightly better	Easiest solution
type #"H type #"e type #"l type #"l type #"o	string #"H #"e #"l #"l #"o	string *#" H e l l o

1-time cat:

```
Method main
char #0
type 1
Method exit
```

Infinite loop:

```
Method main
goto #0
Method exit
```

Hello World (with linefeed):

```
string @"_:32 @":10 *#" H e l l o _ W o r l d ! ^
[
```

Instruction list:

1. Print
2. Type
3. Int
4. Char
5. Goto
6. Match
7. Compare
8. End
9. Transport
10. Shift
11. Add
12. Return
13. Output
14. Skip
- 15.

0	null	none	8	compare	flow
1	print	out	9	end	flow
2	type	out	10	transport	data
3	string	out	11	shift	data
4	int	in	12	add	data
5	char	in	13	return	method
6	goto	flow	14	output	method
7	match	flow	15	skip	method

Not included: modify, printMethod (maybe add -, /, * methods too?)